Java reflection basics

Reflection is the ability to examine programs at runtime. For example, we can load a class, look at the methods and fields in the class, get the bytecode inside a method etc, all at runtime, using a simple Java program! This is a powerful feature in Java, and we are going to use it to get information about classes that we can use for graphing.

If you're from a *C/C++* background, reflection will be very new to you, since those languages don't support this feature.

Here is our first Java program using reflection. Given the name of a class on the command line, the program will load that class dynamically, and print the class name and its super-class name:

```
// BaseDerived.java file
class BaseDerived {
    public static void main(String []args) throws Exception {
      if(args.length != 1) {
        System.err.println("Give the name of the class as the argument");
        System.exit(-1);
       }
      Class currentClass = Class.forName(args[0]);
        System.out.println("passed class is: " + currentClass);
      System.out.println("its base class is: " + currentClass.
getSuperclass());
    }
}
$ javac BaseDerived.java
$ java BaseDerived java.util.LinkedList
passed class is: class java.util.LinkedList
its base class is: class java.util.AbstractSequentialList
$
```

The *Class* class has a method called *forName*—we can use it to search for classes and load them at runtime. We can give any class name; we'll do a lookup for a class named *java.util.LinkedList* here. If the JVM is not able to find the class, it will throw an exception. We will not handle the exception in our program, and declare that main will throw that exception. If the JVM is able to find that class, the *currentClass* variable will point to the *LinkedList* class that is loaded into memory. We can do a lot of things with this *currentClass* variable. For example, we can get the list of methods in the class, and get the super-class name (using *getSuperclass()*) and print the information.

Printing the class hierarchy

Now we're ready to print the whole hierarchy of a given class. We'll use *getSuperClass()* to get the base class in a loop, till we reach the *Object* class. If we try to get the base class of the Object class, we get *null* returned, so we can use it as the termination condition for the loop:

```
// ClassHierarchy.java file
class ClassHierarchy {
      public static void main(String []args) throws Exception {
            if(args.length != 1) {
            System.err.println("Give the name of the class as the argument");
            System.exit(-1);
            }
            Class currentClass = Class.forName(args[0]);
        while(currentClass != null) {
              System.out.println(currentClass);
              currentClass = currentClass.getSuperclass();
            }
      }
}
```

Here is the output of running the program:

```
$ java ClassHierarchy java.util.LinkedList
class java.util.LinkedList
class java.util.AbstractSequentialList
class java.util.AbstractList
class java.util.AbstractCollection
class java.lang.Object
$
```

The output is fine, except for the order of the classes—it prints from the leaf class to the base class, and we want it in the reverse order. If we use recursion, as in this *checkBaseClass* method, we'll get it in the correct order:

```
      private static void checkBaseClass(Class currentClass) {
            if(currentClass != null) {
                checkBaseClass(currentClass.getSuperclass());
                System.out.println(currentClass);
                }
        }
```

Class hierarchy in Graphviz

Now let us write a program, *FullClassHierarchy.java* to output a *dot* file, so we can then print the hierarchy visually. What we need to do is create the proper links in the nodes inside a digraph entry. We'll add/modify *printf* statements for the class relationships. We'll also take one more step, and print the interfaces implemented by classes in the *LinkedList* hierarchy.

(http://www.linuxforu.com/article_source_code/june10/graphviz/ClassHierarchy.java *)*

```
import java.util.*;
class FullClassHierarchy {
      private static void checkInterfaces(Class currentClass) {
        if(currentClass != null) {
            Class []interfaces = currentClass.getInterfaces();
            for(Class anInterface : interfaces) {
                System.out.printf(" \"%s\" -> \"%s\" [style=\"dotted\"] \n",
                anInterface.getName(), currentClass.getName());
            }
        }
      }
```